

Полная шпаргалка Python для новичка (максимально подробная)

Эта версия расширенная: синтаксис, ввод/вывод, числа, строки, списки, кортежи, циклы for/while, условия, вложенные структуры, enumerate(), break/continue/else, срезы, List Comprehension, типовые шаблоны решений и частые ошибки.

1) Структура программы: как решать любую задачу

```
# Универсальный план решения задачи:
# 1) Прочитать условие и понять, что нужно получить в конце
# 2) Ввести данные (input) или задать их в коде
# 3) Подготовить переменные (накопители, списки, счетчики)
# 4) Написать основную логику (if / for / while)
# 5) Вывести результат (print)

# Пример каркаса:
# a = int(input())
# b = int(input())
# result = a + b
# print(result)
```

2) Комментарии, отступы и блоки кода

Отступы — это часть синтаксиса Python. Если отступы неправильные, программа не запустится.

```
# Комментарий начинается с символа # и игнорируется Python.

# В Python блоки кода задаются ОТСТУПАМИ (обычно 4 пробела).
# Пример:
x = 10
if x > 0:
    print("x положительный")
    print("Эта строка тоже внутри if")
print("А эта строка уже снаружи if")
```

3) Переменные: создание и правила именования

```
# Переменная — это имя, которое хранит значение.
name = "Миша"
age = 40

# Правила именования:
my_name = "Alex"      # можно
_age = 25              # можно
# lname = "Ошибка"    # нельзя (нельзя начинать с цифры)

# Регистр важен:
Name = "Один"
name = "Другой"
print(Name) # Один
print(name) # Другой
```

4) Типы данных: int / float / str / bool + type()

```
# Основные типы:
a = 10          # int (целое)
b = 3.14        # float (дробное)
c = "Python"    # str (строка)
d = True        # bool (логика)

print(type(a))  # <class 'int'>
print(type(b))  # <class 'float'>
print(type(c))  # <class 'str'>
print(type(d))  # <class 'bool'>
```

5) Преобразование типов: int(), float(), str(), bool()

Важно: input() всегда возвращает строку, поэтому часто нужен int() или float().

```
# Преобразование типов нужно, чтобы работать с данными правильно.
x = "123"
```

```

print(int(x) + 7)      # 130

y = "3.5"
print(float(y) * 2)    # 7.0

n = 100
print(str(n) + " тенге") # "100 тенге"

# bool() – True для непустых значений и False для пустых/0:
print(bool(0))         # False
print(bool(1))         # True
print(bool(""))        # False
print(bool("hi"))      # True

```

6) Ввод данных: input() (главная ловушка новичка)

```

# input() всегда возвращает строку (str)
x = input("Введите число: ")
print(type(x)) # <class 'str'>

# Чтобы получить число:
a = int(input("Введите целое число: "))
b = float(input("Введите дробное число: "))

print(a + 10)
print(b * 2)

```

7) Вывод данных: print() + sep + end

```

# print() может печатать несколько значений:
name = "Миша"
age = 40
print("Имя:", name, "Возраст:", age)

# sep – чем разделять значения (по умолчанию пробел):
print("A", "B", "C", sep=" | ") # A | B | C

# end – что печатать в конце строки (по умолчанию перенос строки \n):
print("Привет", end=" ")
print("мир!") # Привет мир!

```

8) Специальные символы в строках (\n, \t, \\\, \")

```

print("Первая строка\nВторая строка") # \n перенос строки
print("A\tB\tC")                       # \t табуляция
print("Слэш: \\")                      # \\ вывести символ \
print("Кавычки: \"текст\"")            # \" двойные кавычки

```

9) f-string: вставка переменных и округление

f-string — самый удобный способ форматировать строки в Python.

```

name = "Миша"
age = 40
print(f"Привет, {name}! Тебе {age} лет.")

pi = 3.1415926
print(f"pi = {pi:.2f}") # 2 знака после запятой -> 3.14

# Можно вставлять выражения:
a = 7
b = 5
print(f"{a} + {b} = {a + b}")

```

10) Арифметические операторы: + - * / // % **

```

a = 15
b = 4

print(a + b) # 19
print(a - b) # 11
print(a * b) # 60
print(a / b) # 3.75 (деление всегда float)

```

```
print(a // b) # 3      (целая часть)
print(a % b)  # 3      (остаток)
print(a ** b) # 50625 (степень)
```

11) Приоритет операций (скобки решают всё)

Если сомневаешься — ставь скобки. Это делает код понятнее.

```
# Умножение и деление выполняются раньше сложения:
print(2 + 3 * 4)      # 14

# Скобки меняют порядок:
print((2 + 3) * 4)    # 20
```

12) Операторы сравнения: > < == != >= <=

```
x = 7
y = 10

print(x > y)   # False
print(x < y)   # True
print(x == y)  # False
print(x != y)  # True
print(x >= 7)  # True
print(y <= 9)  # False
```

13) Логические операторы: and / or / not

```
age = 20
is_student = True

print(age > 18 and is_student) # True (оба True)
print(age < 18 or is_student)  # True (хотя бы одно True)
print(not is_student)          # False (инверсия)
```

14) Условия if / else (2 варианта)

```
age = 17

if age >= 18:
    print("Доступ разрешен")
else:
    print("Доступ запрещен")
```

15) Условия if / elif / else (много вариантов)

```
score = 77

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
elif score >= 60:
    print("D")
else:
    print("F")
```

16) Вложенные условия (if внутри if)

Вложенные условия полезны, но не делай слишком глубокую 'пирамиду' из if.

```
age = 25
has_ticket = True
has_passport = False

if age >= 18:
    print("Взрослый")
    if has_ticket:
        print("Есть билет")
        if has_passport:
            print("Можно лететь")
```

```

        else:
            print("Нужен паспорт")
    else:
        print("Нет билета")
else:
    print("Несовершеннолетний")

```

17) Тернарный оператор (условие в одну строку)

```

age = 19
status = "Взрослый" if age >= 18 else "Ребёнок"
print(status)

```

18) Цикл for: перебор значений

```

# Перебор строки:
word = "Python"
for ch in word:
    print(ch)

# Перебор списка:
nums = [10, 20, 30]
for x in nums:
    print(x)

```

19) range(): 1, 2 или 3 аргумента

```

# range(stop) -> 0..stop-1
for i in range(5):
    print(i) # 0 1 2 3 4

# range(start, stop) -> start..stop-1
for i in range(1, 6):
    print(i) # 1 2 3 4 5

# range(start, stop, step)
for i in range(2, 11, 2):
    print(i) # 2 4 6 8 10

```

20) Обратный range() (шаг -1)

```

for i in range(10, 0, -1):
    print(i) # 10 9 8 ... 1

```

21) Накопление суммы в цикле (шаблон)

Накопитель — одна из главных идей в программировании.

```

total = 0
for i in range(1, 6):
    total += i # total = total + i
print(total) # 15

```

22) Поиск максимума/минимума вручную (шаблон)

```

nums = [5, 2, 9, 1, 7]

max_value = nums[0]
for x in nums:
    if x > max_value:
        max_value = x

print("Максимум:", max_value)

```

23) Цикл while: когда неизвестно число повторений

```

i = 0
while i < 5:
    print(i)
    i += 1

```

24) Бесконечный while True + break (правильный выход)

```
while True:
    text = input("Введите stop для выхода: ")
    if text == "stop":
        break
    print("Вы ввели:", text)
```

25) break и continue: управление циклом

```
# break – выйти из цикла полностью
for i in range(10):
    if i == 5:
        break
    print(i)

# continue – пропустить текущую итерацию
for i in range(5):
    if i == 2:
        continue
    print(i)  # 0 1 3 4
```

26) else у циклов (for/while ... else)

```
# else выполнится только если цикл завершился без break
for i in range(5):
    if i == 10:
        break
else:
    print("break не сработал, цикл завершился сам")
```

27) Списки: создание, индексы, изменение

```
lst = [10, 20, 30]
print(lst[0])  # 10
print(lst[-1]) # 30

lst[1] = 999
print(lst)     # [10, 999, 30]
```

28) Методы списков: append / extend / insert

```
data = [1, 2, 3]

data.append(4)      # добавить 1 элемент
data.extend([5, 6]) # добавить несколько элементов
data.insert(0, 0)   # вставить по индексу

print(data)
```

29) Удаление из списка: remove / pop / clear / del

```
items = ["a", "b", "b", "c"]

items.remove("b")  # удалит первый 'b'
print(items)

x = items.pop()    # удалит последний и вернет его
print("Удалили:", x)
print(items)

items.clear()      # очистить список
print(items)

nums = [10, 20, 30, 40]
del nums[1]        # удалить по индексу
print(nums)
```

30) Поиск в списке: index / count / in

```
data = [5, 10, 10, 20]
print(data.index(10)) # 1
```

```

print(data.count(10))    # 2

print(20 in data)        # True
print(99 in data)        # False

```

31) Сортировка и разворот: sort / reverse / sorted()

```

nums = [3, 1, 2]

nums.sort()
print(nums)    # [1,2,3]

nums.sort(reverse=True)
print(nums)    # [3,2,1]

nums.reverse()
print(nums)    # [1,2,3] (разворот текущего порядка)

# sorted() возвращает новый список, не меняя исходный:
a = [5, 2, 9]
b = sorted(a)
print(a)    # [5,2,9]
print(b)    # [2,5,9]

```

32) Копирование списков: copy() и ошибка с =

```

a = [1, 2, 3]

b = a          # ОПАСНО: b и a — это один и тот же список
b.append(999)
print(a)       # [1,2,3,999] (изменился a!)

# Правильно:
a = [1, 2, 3]
b = a.copy()   # новый список
b.append(999)
print(a)       # [1,2,3]
print(b)       # [1,2,3,999]

```

33) Срезы списков: start:stop:step

```

data = [0, 1, 2, 3, 4, 5, 6]

print(data[1:4])    # [1,2,3]
print(data[:3])     # [0,1,2]
print(data[3:])     # [3,4,5,6]
print(data[::2])    # [0,2,4,6]
print(data[::-1])   # разворот

```

34) enumerate(): индекс + значение

```

items = ["A", "B", "C"]

for i, value in enumerate(items):
    print(i, value)

for num, value in enumerate(items, start=1):
    print(f"№{num}: {value}")

```

35) Вложенные списки (матрицы): доступ и перебор

```

matrix = [
    [1, 2, 3],
    [4, 5, 6]
]

print(matrix[0])    # первая строка
print(matrix[1][2]) # 6

for row in matrix:
    for value in row:
        print(value, end=" ")
    print()

```

36) Вложенные циклы: таблица умножения

```
for row in range(1, 6):
    for col in range(1, 6):
        print(f"{row*col:<3}", end="")
    print()
```

37) Кортежи (tuple): что это и зачем

```
t = (1, 2, 2, 3)

print(t[0])      # 1
print(t.count(2)) # 2
print(t.index(3)) # 3

# Кортеж нельзя изменить:
# t[0] = 999 # будет ошибка

# Если нужно изменить – делаем через list:
lst = list(t)
lst.append(999)
t2 = tuple(lst)
print(t2)
```

38) Строки (str): индексы, срезы, базовые операции

```
text = "Python"

print(text[0])    # 'P'
print(text[-1])   # 'n'
print(text[1:4])  # 'yth'
print(text[::-1]) # разворот

print("Py" + "thon") # склейка строк
print("ha" * 3)      # повтор -> hahaha
```

39) Методы строк: upper/lower/strip/replace/find/split/join

```
s = " Hello, World! "
```

```
print(s.upper())
print(s.lower())
print(s.strip())          # убрать пробелы по краям

print(s.replace("World", "Python"))

print(s.find("Hello"))    # индекс или -1
print(s.count("l"))       # сколько раз

tags = "python,code,study"
lst = tags.split(",")
print(lst)

joined = " | ".join(lst)
print(joined)
```

40) List Comprehension: создание списков в одну строку

```
# Базовый шаблон:
squares = [x**2 for x in range(1, 6)]
print(squares)

# Фильтрация:
evens = [x for x in range(1, 11) if x % 2 == 0]
print(evens)

# if-else внутри:
labels = ["чет" if x % 2 == 0 else "нечет" for x in range(1, 6)]
print(labels)

# Вложенный comprehension (матрица):
matrix = [[i*j for j in range(1, 4)] for i in range(1, 4)]
```

```
print(matrix)
```

41) Частые шаблоны задач (готовые скелеты)

Эти шаблоны можно копировать и подставлять свои данные/условия.

```
# 1) Сумма чисел от 1 до N
n = int(input("N: "))
total = 0
for x in range(1, n + 1):
    total += x
print(total)

# 2) Сумма четных от 1 до N
n = int(input("N: "))
total = 0
for x in range(1, n + 1):
    if x % 2 == 0:
        total += x
print(total)

# 3) Подсчет элементов по условию
nums = [1, 5, 8, 2, 10]
count = 0
for x in nums:
    if x > 5:
        count += 1
print(count)

# 4) Поиск первого подходящего элемента
nums = [1, 5, 8, 2, 10]
for x in nums:
    if x > 7:
        print("Нашли:", x)
        break
```

42) ТОП ошибок новичка и как исправлять

```
# Ошибка 1: input() без int/float
# a = input()
# print(a + 5) # ошибка
a = int(input())
print(a + 5)

# Ошибка 2: путаница = и ==
x = 10
print(x == 10) # сравнение
x = 20         # присваивание

# Ошибка 3: забыли про диапазон range (stop не включается)
for i in range(5):
    pass # будет 0..4

# Ошибка 4: неверная логика с or
score = 75
# if score >= 90 or score >= 70: # почти всегда True
# правильно:
if score >= 90:
    print("A")
elif score >= 70:
    print("B/C")
```